

PROYECTO FINAL

Generación de arte ASCII mediante procesamiento paralelo en GPU

CC7515-1 – Computación en GPU 2026

Marcelo Chuang Y. · Oscar Garrido O.



El problema y su objetivo

¿Qué es el arte ASCII?

- Representación de una imagen mediante caracteres de texto.
- Cada carácter tiene una densidad visual distinta (más o menos "tinta").
- Regiones oscuras y claras se representan con caracteres diferentes.
- Cada carácter reemplaza a un conjunto de píxeles de la imagen original.

Motivación

- Problema visual, fácil de demostrar y de verificar.
- Operaciones independientes por región de la imagen.
- Permite estudiar paralelismo de datos en GPU.
- Permite medir costos de transferencia y ejecución de kernels.

Objetivo principal

- Implementar y comparar la conversión de imágenes a arte ASCII: una versión secuencial en CPU y una versión paralela en GPU con OpenCL.

Principales desafíos

- Diseñar una escala ASCII coherente con la densidad visual percibida por la luminosidad.
- Manejar bloques de borde cuando la imagen no es divisible exactamente.

Fundamento teórico del algoritmo

LUMINOSIDAD DE UN PÍXEL

$$L = 0.299 R + 0.587 G + 0.114 B$$

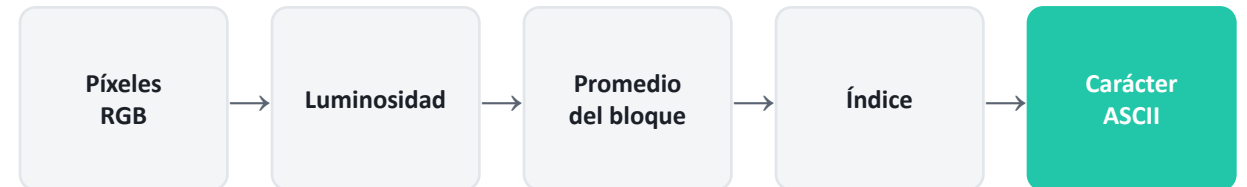
L queda aproximadamente en el rango [0, 255]

ESCALA ASCII UTILIZADA

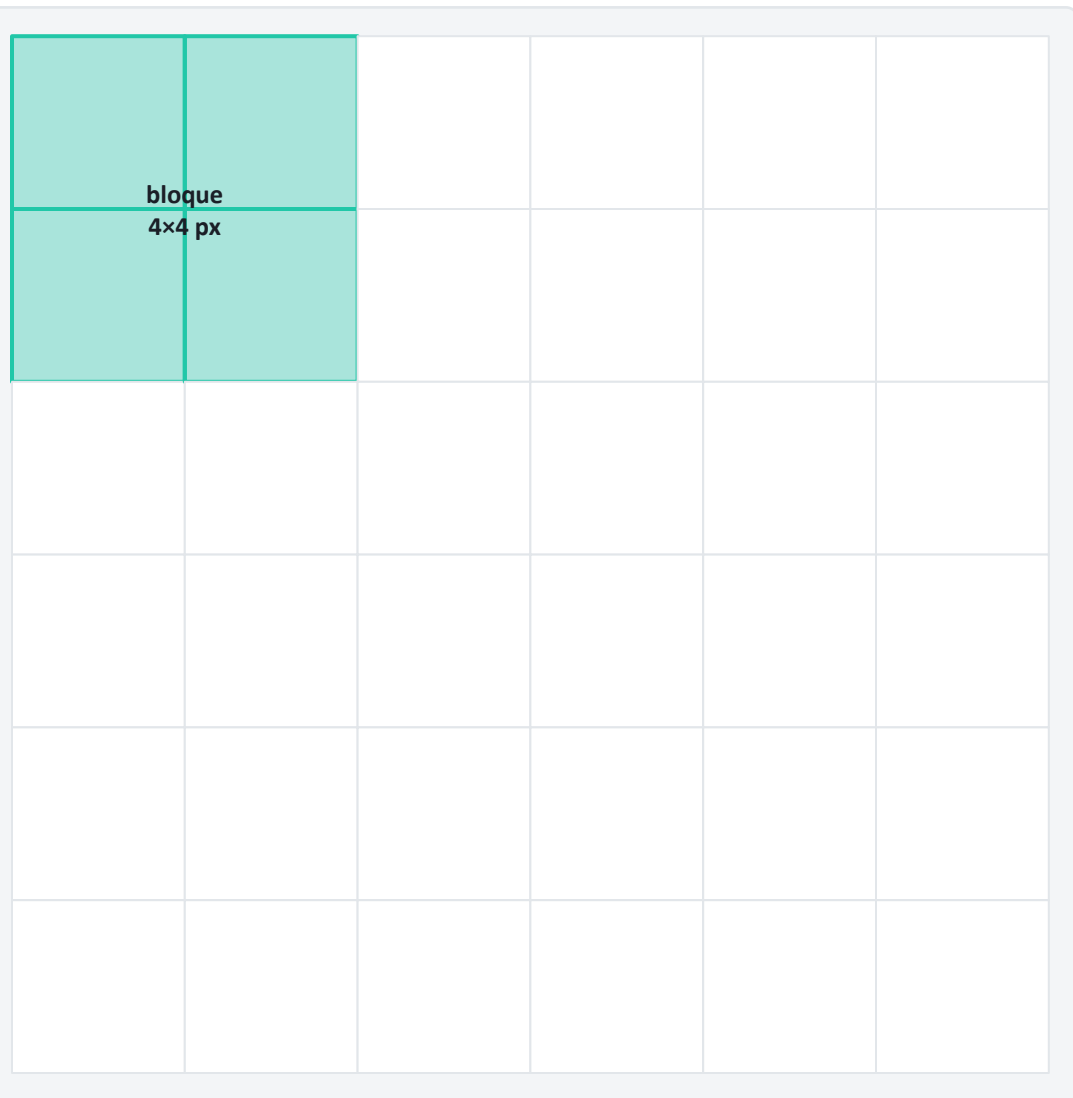
[. : - = + * # % @]

¿Por qué esos pesos?

- El verde pesa más porque la percepción humana es más sensible a ese componente.
- Se calcula la luminosidad promedio de todos los píxeles de un bloque.
- El promedio se transforma en un índice dentro de una escala de caracteres.
- 0 → negro, 255 → blanco (a confirmar el carácter exacto de cada extremo).



División de la imagen en bloques



Parámetros configurables

- `--block-width` · `--block-height`
- Valores predeterminados actuales: 4 × 4 píxeles.

```
output_width =  
    (image_width + block_width - 1)  
    / block_width;  
  
output_height =  
    (image_height + block_height - 1)  
    / block_height;
```

Efecto del tamaño de bloque

- Bloques pequeños: más detalle, más caracteres. Bloques grandes: menos detalle.
- Bordes con menos píxeles si la imagen no es divisible exactamente por el bloque.
- Terminales no son cuadradas: bloques como 8×16 preservan mejor la proporción.

Detalles de la solución

Stack tecnologico

C++20

OpenCL 3.0

stb_image

argparse

Carga de imágenes con stb_image

- Carga la imagen en CPU como un arreglo RGB intercalado:
R G B | R G B | R G B | R G B | ...
- Se fuerza la carga a 3 canales (RGB).



Implementación en CPU

CPU

Firma de la función

- Entradas: píxeles RGB, ancho y alto de imagen, ancho y alto de bloque.
- Salida: objeto AsciiArt con ancho y alto y caracteres ascii.

```
AsciiArt run_cpu(const unsigned char *image,
                 int image_width,
                 int image_height,
                 int block_width,
                 int block_height)
```

```
para cada fila de bloques:
  para cada columna de bloques:
    suma = 0; cantidad = 0
    para cada píxel válido del bloque:
      L = luminosidad(píxel)
      suma += L; cantidad++
    promedio = suma / cantidad
    carácter = escala[indice(promedio)]
    guardar carácter
```

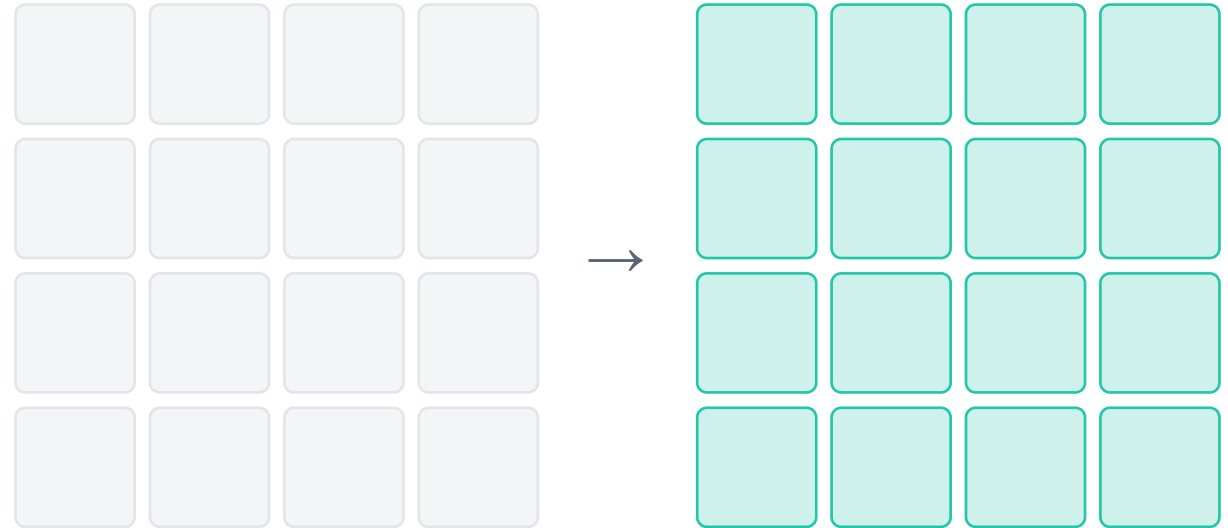
¿Por qué el problema es paralelizable?

GPU

Independencia entre bloques

- Cada bloque puede procesarse de forma completamente independiente.
- Solo se necesita: la posición del bloque, sus píxeles y la escala ASCII.
- No es necesario conocer el resultado de ningún otro bloque.
- Se asigna un work-item OpenCL distinto a cada celda de la salida ASCII.

Cuadrícula de bloques → cuadrícula de work-items OpenCL



Implementación en OpenCL

1. Inicializar OpenCL

2. Seleccionar dispositivo

3. Crear contexto

4. Crear command queue

5. Compilar ascii-gray.cl

6. Buffer de imagen RGB

7. Buffer de salida ASCII

8. Configurar argumentos

9. Ejecutar kernel 2D

10. Leer resultado

Buffers

- image_buffer — lectura — contenido: imagen RGB.
- ascii_buffer — escritura — contenido: un carácter por bloque.

```
ClRunner cl;  
cl.init();  
  
cl_kernel ascii_kernel =  
cl.load_program("../kernel/ascii-gray.cl", "ascii");  
  
cl_mem image_buffer = cl.create_buffer(  
    image_size,  
    CL_MEM_READ_ONLY |  
    CL_MEM_COPY_HOST_PTR,  
    image  
);  
  
cl_mem ascii_buffer = cl.create_buffer(  
    output_size * sizeof(cl_uchar),  
    CL_MEM_WRITE_ONLY  
);  
  
cl.set_args()...
```


Organización de work-items y work-groups en kernel

NDRange bidimensional

- `global_id(0)` = columna del bloque
- `global_id(1)` = fila del bloque
- Cada work-item calcula un carácter de la salida ASCII.

```
// work-group: 16 x 16 = 256 work-items
size_t local_w = 16;
size_t local_h = 16;
```

```
global_w = (output_width + local_w - 1)
           / local_w * local_w;
global_h = (output_height + local_h - 1)
           / local_h * local_h;
```

El tamaño global se redondea al múltiplo de 16 superior: pueden lanzarse work-items adicionales fuera de la salida válida.

```
__kernel void ascii(
    __global const uchar* image,
    __global char* ascii,
    int img_w, int img_h,
    int block_w, int block_h,
    int zones_w, int zones_h
)

x = get_global_id(0);
y = get_global_id(1);

si x o y están fuera de la salida:
    retornar

por cada píxel válido del bloque:
    L = luminosidad(píxel)
    suma += L; cantidad++
promedio = suma / cantidad
carácter = escala[indice(promedio)]
guardar carácter
```

Validación y metodología experimental

Proceso de validación

- Ejecutar CPU y OpenCL sobre la misma imagen.
- Mismo tamaño de bloque, misma fórmula de luminosidad y misma escala ASCII.
- Comparar: ancho, alto, cantidad de caracteres y cada carácter individual.

RESULTADO ESPERADO

Diferencias CPU / OpenCL = 0

Metodología de rendimiento

- Ejecutar varias repeticiones con las mismas imágenes y parámetros.
- En OpenCL, separar: inicialización, compilación, procesamiento

```
// CPU
inicio = steady_clock::now();
ascii  = run_cpu(...);
fin    = steady_clock::now();

// OpenCL
inicio_init = steady_clock::now();
cl.init();
end_init, inicio_setup = steady_clock::now();
cl.load_program(...);
end_init, inicio_setup = steady_clock::now();
cl.create_buffer();
cl.set_argument();
ascii  = cl.run_kernel;
end_init, inicio_setup = steady_clock::now();
```

DEMO

Resultados y conclusiones

Tabla de resultados

Resolución	Bloque	CPU (ms)	OpenCL init (ms)	GPU proc. (ms)	GPU setup (ms)
600x528	4 × 4	2.3	161	0.57	3.1
600x528	8 × 8	1.9	163	0.57	3.1
967x959	8 × 8	5.8	160	1.7	3.1

Conclusiones y trabajo futuro

- El problema es paralelizable: los bloques se procesan de forma independiente.
- OpenCL distribuye los bloques entre múltiples work-items.
- En imágenes pequeñas, el costo de setup/transferencia puede superar el beneficio de la GPU.
- Trabajo futuro: color, video/animación, reutilizar contexto OpenCL, otros tamaños de work-group.

Bibliografía

- <https://en.wikipedia.org/wiki/Grayscale>
- <https://github.com/DLYT-Dev/Image2Ascii>